Latent Diffusion World Models for Grid Games

Learning (frame, action) $\rightarrow$ next-frame across 10 games

Spatial-Intel

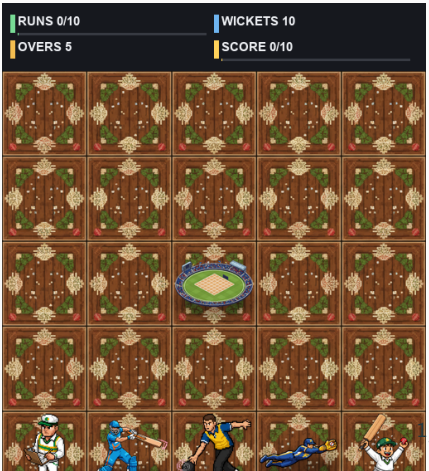
August 3, 2026

DIAMOND denoiser + Qwen-Image VAE • 30,323 transitions

The problem

Goal. A single world model that, given a game frame and an action, predicts the next frame: learned from data, across many games, not scripted.

- Input: current frame + discrete action (W/A/S/D/Space).
- Output: next frame (player moved, HUD updated, scene preserved).
- Hard part: the moving player is $<1\%$ of the frame; whole-frame loss is dominated by the static 99%.



Data

The dataset: 10 games, one action space

game	train	val
hospital	6816	1704
shopping_mall	4887	1222
wildlife	4264	1066
crops_and_farming	2508	627
7_worlds_of_wonder	2112	528
space_exploration	1293	323
eid	1050	262
cricket	904	226
diwali	262	65
reduce_reuse_recycle	163	41
total	**24,259**	**6,064**

Unified action space (5).
All games map to `W/S/A/D/Space`. Every game's dozens of context-actions (TreatPatient, RescueTiger, ...) are `Space`. **Split.** Stratified 80/20 per game: every game appears in both train *and* val, so validation measures in-distribution generalization for all 10 games. Each row: `(obs_t, action, obs_next)` + effects (player move, HUD changes). Clean deterministic renders.

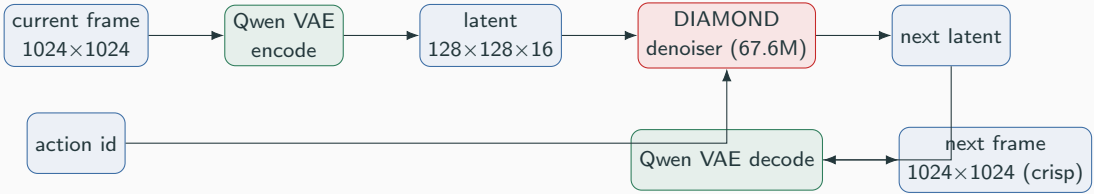
State space & win conditions per game (from RDDL)

game	grid	states	trans.	actions	**goal states**	opt. route	max rew.
hospital	4×4	2640	8520	9	720	10	8
shopping_mall	7×7	3001	6109	12	72	11	33
wildlife	5×5	2970	5330	17	957	9	82
crops_and_farming	6×6	1499	3135	9	score-based	n/a	40
7_worlds_of_wonder	5×5	800	2640	9	25	16	10
space_exploration	4×4	512	1616	9	16	14	7
eid	5×5	400	1312	8	25	12	6
cricket	5×5	275	1130	5	25	10	10
diwali	5×5	100	327	7	25	7	3
reduce_reuse_recycle	3×3	72	204	7	9	6	3
total		**12,269**	**30,323**				

- **states** = reachable board configurations; **goal states** = winning configurations (RDDL goal predicate); **opt. route** = shortest action sequence to win.
- Wide range: hospital 2640 states, r_r_r just 72. The tiny games have few states *and* few transitions: this is the data floor.

Architecture

Architecture: latent diffusion, not pixel



- Why latent? Pixel-space attention at 1024px needs ∼1.5 TB (quadratic in tokens): OOM. The VAE compresses 8×; attention runs only at 32×32 / 16×16 (SD / GameNGen pattern).
- VAE round-trip is near-lossless (MAE 2.5/255): the *decode* is not the bottleneck: the *denoiser*'s prediction is.

The journey

What we found: the model was copying the input

First runs hit low loss (val ≈ 0.03): but decoding revealed the predictions just reproduced the input frame.

Root cause: loss dilution
The change per transition is tiny ($|next - cur| \approx 0.01$ in latent). Plain MSE is dominated by the static 99% of the frame, so copying the input already scores near-zero loss. The model never learns to move the player.

Measured **change-region accuracy** (does the prediction match *next* better than the copied *input*, in the pixels that changed):

$$\text{plain MSE} \Rightarrow \sim 0\% \quad \text{(pure copy)}$$

Fix 1+2: change-weighted loss + stronger conditioning

Change-weighted loss (in DIAMOND's EDM `forward`, only the loss reduction changed):

$$w = 1 + K\,|z_{next} - z_{cur}|, \quad \mathcal{L} = \left(w\,(\hat{v} - v)^2\right)_{\text{mean}}$$

upweights the region that actually moves (K=20). Penalizes copying. **Stronger action conditioning:** `cond_channels` 256→512. DIAMOND injects the action embedding into every UNet block; a larger cond vector makes the action harder to ignore.

change-acc

0% → 26%

copy-bug fixed; one game hit 73%, but tiny games still 0%

Fix 3: game-balanced sampling

Problem. hospital has 6,816 train transitions; `reduce_reuse_recycle` has 163. Uniform sampling picks hospital $\sim 40\times$ more: tiny games are drowned out and never learn (0% change-acc). **Fix.** Sample game-first (uniform over games, then a transition): every game gets equal training exposure.

change-acc	plain MSE	+ weighted	+ balanced
overall	$\sim$0%	26%	**75%**
diwali (262)	0%	0%	27%
reduce_reuse (163)	0%	0%	40%
7_worlds (2112)	0%	73%	100%

Results

Result: 0% → 75% change-accuracy

game	change-acc	game	change-acc
7_worlds_of_wonder	100%	hospital	93%
shopping_mall	100%	wildlife	87%
cricket	93%	eid	60%
crops_and_farming	93%	space_exploration	60%
reduce_reuse_recycle	40%	diwali	27%

- The copy-bug is fixed: the model predicts the move, doesn't copy.
- Tiny games (diwali, r_r_r) still lag: residual data-quantity floor (∼160 to 260 train transitions each).

But: the moving player renders as a clean blur



shopping_mall: input | predicted | ground truth

Background & HUD: crisp. Tiles, counters, static objects reconstruct sharply. **Moving player: a soft haze**, not a sprite. The model knows the *region* but hedges the exact pixels. Why (fundamental): a deterministic predictor minimizing loss returns the *mean* over uncertain sprite placements: and the mean of many sharp sprites is a blur. Change-weighting (K=20 $\rightarrow$ 40, fine-tune) removed artifacts but did not sharpen this.

Loss changes → effect on val images (summary)

configuration	val loss	change-acc	decoded image
plain MSE, 512px	0.03	∼0%	crisp bg, player copied
plain MSE, 1024px	0.29†	∼0%	sharper bg, still copies
+ change-weighted (K20)	0.15	26%	smeary player, ghosts
+ balanced	0.17	75%	player moves, blurry
+ sharpen fine-tune (K40)	0.19	75%	ghosts gone, still soft

† higher because undertrained at 1024px; loss values are *not* comparable across configs: change-weighting deliberately raises loss by counting the moved region. change-acc is the honest metric.

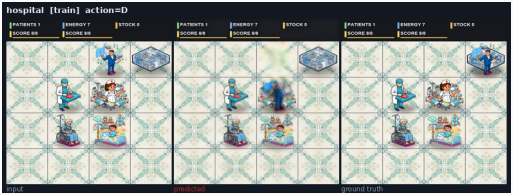
Prediction gallery

Prediction gallery: input | predicted | ground truth

Below: the world model on held-out (val) and train transitions, across all 10 games. Backgrounds/HUD reconstruct crisply; the moving player is the soft region. Each strip is one `(frame, action)` $\rightarrow$ next-frame.

hospital

train



held-out val



shopping_mall

train



held-out val



wildlife

train

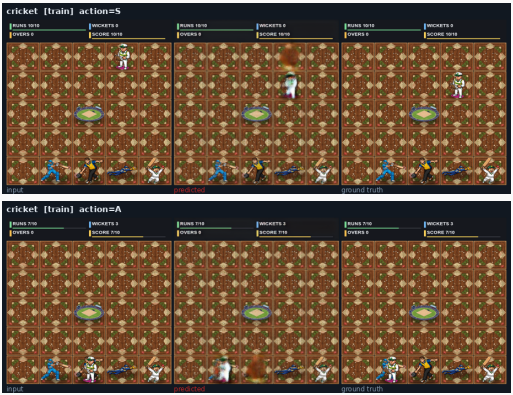


held-out val

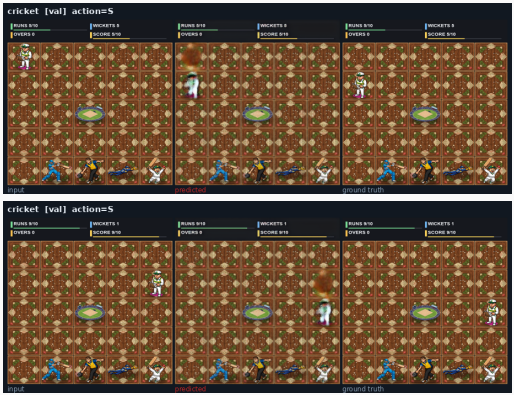


cricket

train

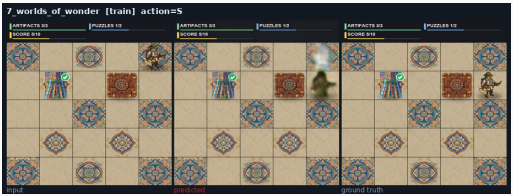


held-out val

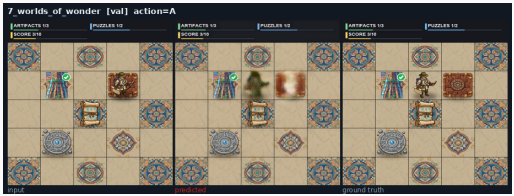


7_worlds_of_wonder

train

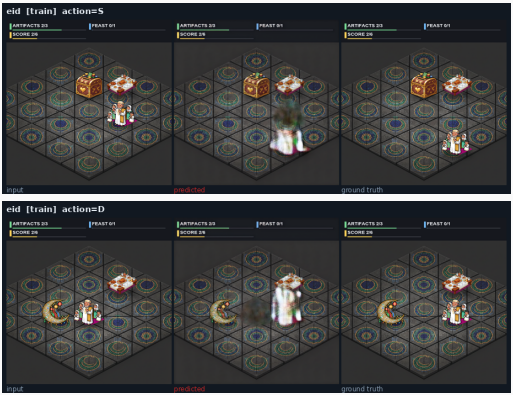


held-out val

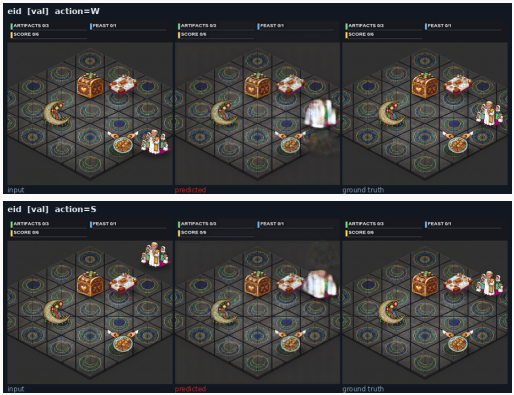


eid

train

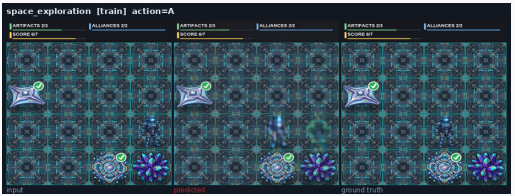


held-out val

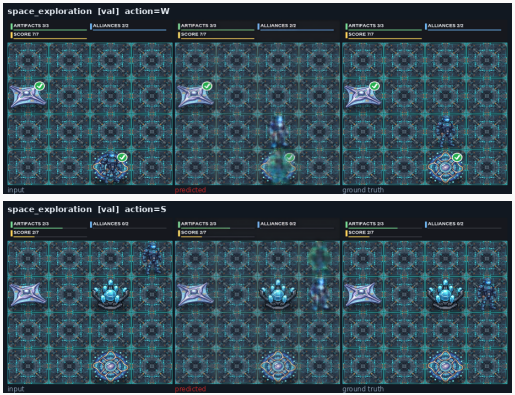


space_exploration

train

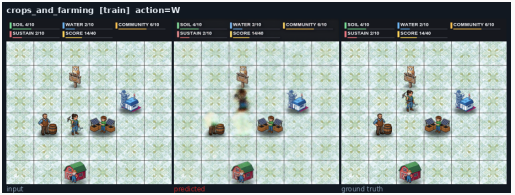


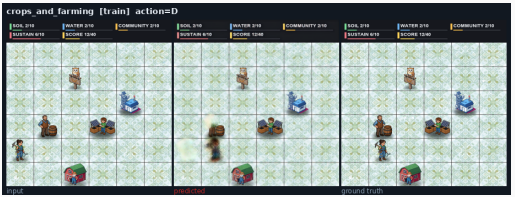
held-out val



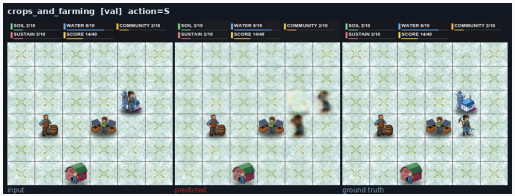
crops_and_farming

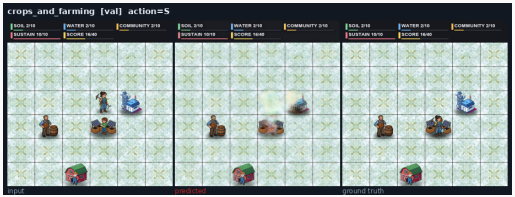
train





held-out val





diwali

train

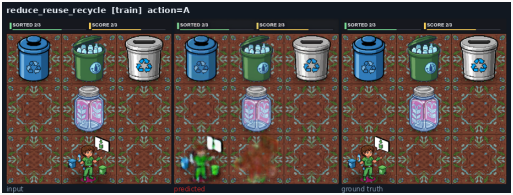


held-out val

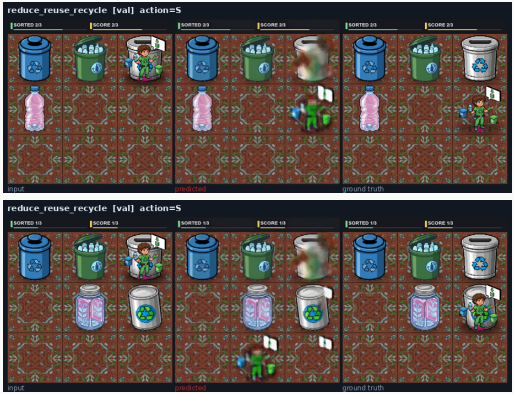


reduce_reuse_recycle

train



held-out val



Making the WM sharper

This blur is a known phenomenon

Our diagnosis matches the literature exactly: action-conditioned diffusion world models suffer regression-to-the-mean blur.

The distortion-perception tradeoff (fundamental)
A predictor minimizing a point loss (MSE) returns the conditional mean : the average of all plausible next-frames. The mean of many sharp sprites *is* a blur. You cannot minimize distortion *and* be sharp; they are provably at odds (Blau & Michaeli).

- This is why our change-acc (a distortion metric) reads 75% while the image looks soft: distortion metrics *reward* the blurry mean.
- AV world-model papers report the same: regressors win SSIM with a blurry mean; diffusion wins distribution metrics (KID 4.8× better).

How to make the WM sharper: three levers

1. **Stochastic sampling** (no retrain). We sampled with a *deterministic* solver, which converges to the mean. A stochastic (SDE / churn) sampler commits to a *mode* instead of averaging: the standard cure for over-smooth output. Try first: free.
2. **A distribution loss, not just MSE.** Add an adversarial / perceptual (LPIPS) term so the model is scored on *realism*, not pixel-distance: moves it off the conditional mean toward a sharp sample.
3. **Capacity + data.** 67.6M params over 10 games is thin; a larger UNet trained longer can render sprite detail. For the tiny games (diwali, r_r_r) the real limit is data quantity: a sample-efficient **RSSM (Dreamer)** or synthetic augmentation helps.

Lever 1 is a sampling-time change on our *existing* checkpoint: the cheapest honest next experiment.

We ran Lever 1: stochastic sampling works (free)

Same checkpoint, no retraining. Deterministic sampling converges to the blurry mean; adding stochastic churn makes the character commit to a shape.

sampler	MS-SSIM ↑ distortion	LPIPS ↓ perceptual
deterministic	0.9585	0.1141
stoch churn 20	0.9546	0.1101
stoch churn 40	0.9546	**0.1099**
stoch churn 80	0.9513	0.1129
stoch churn 150	0.9458	0.1174

The two metrics disagree:

- MS-SSIM (distortion) picks deterministic: it *rewards the blur*, as the theory predicts.
- LPIPS (perceptual) picks stochastic c40: it matches the eye.

Winner: moderate churn (c40) = best perceptual quality, tiny distortion cost. Too much churn (c150) adds background noise: worse on both.

Stochastic sampling: visual evidence



input | ground truth | deterministic | stochastic (churn 20 to 150). The moving figure gains structure with

Rich conditioning

Rich conditioning: telling the model where and what

A bare action id (`W/A/S/D/Space`) is too weak: the model must *guess* which cell changes. We inject two extra conditioning streams, both derived from the transition's `effects` (given as input):

WHERE: heatmap (spatial)
Two Gaussian blobs on the 128×128 latent grid: one at the vacated cell, one at the destination cell. Concatenated to the latent channels (16+2=18).

WHAT: structured vector
`action_name`, `target` object, and HUD `int_changes` (score, counters). Embedded and added to DIAMOND's FiLM cond vector, injected into *every* UNet block.

The heatmap is conditioning only: it is *not* part of the predicted target.

What the structured (action) vector actually contains

Per game we build a vocab from the RDDL transitions (hospital shown):

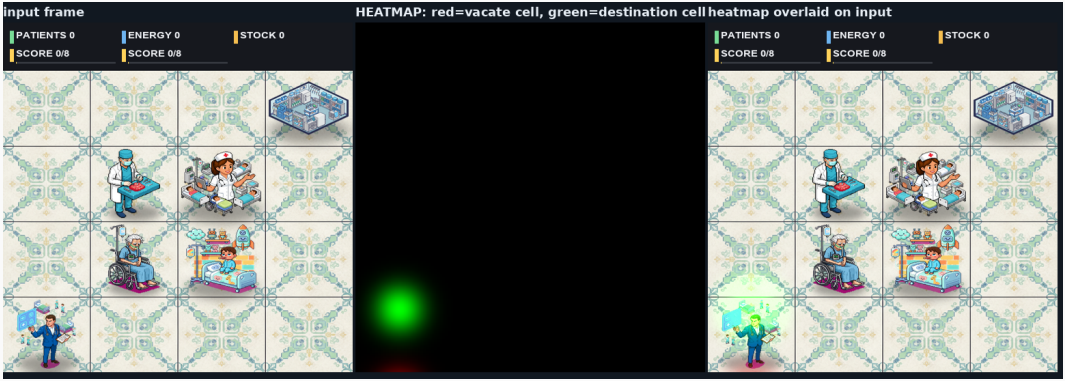
- `action` ∈ {CoordinateStaff, MoveDown, MoveLeft, MoveRight, MoveUp, RestockResources, TreatPatient}
- `target` ∈ {doctor, nurse, patient_1, patient_2, supply_room, ¡none¿}
- `hud` ∈ {patient_treated, resource_stock, staff_energy, total_score, ¡none¿} + `hud_delta` (a signed float)

Each transition ⇒ 3 embedding lookups + 1 scalar, summed:

Example: a move (`W`)
`effects.moved_player:` (1,1)→(1,2)
⇒ action=MoveUp, target=¡none¿,
hud=¡none¿, hud_delta=0.0
(pure movement; heatmap carries the "where")

Example: a treat (`Space`)
`completed_object:` `patient_1`,
`int_changes:` `patient_treated` 0→1
⇒ action=TreatPatient, target=patient_1,
hud=patient_treated, hud_delta=+1.0

What the heatmap looks like



- For move $(1,1) \rightarrow (1,2)$: red = cell the player leaves, green = cell it moves to. Right panel overlays it on the input.
- Built from the per-game `geometry.json` (calibrated cell→pixel map), so it lands *exactly* on the right latent cells. This alone fixed most of the placement error and blur.

Teacher forcing (how we evaluate the renderer)



- Teacher forcing: each step feeds the *real* (or compositor-built) current frame + action; the WM renders the next frame; we re-anchor to the real frame before the next step. Tests the *one-step renderer*.
- Closed loop: feed the model's *own* output forward. Errors compound $\Rightarrow$ drift/blur over long rollouts.
- The compositor (sprites at RDDL cell positions) gives *drift-free* input frames: no reliance on precomputed images $\Rightarrow$ this is a learned renderer, not a lookup table.

Metrics, results & latency

Metrics: undertraining, not a wall

Metric	800 steps	2000 steps	what it measures
SSIM ↑	0.69	0.84	structural similarity
LPIPS ↓	0.26	0.13	perceptual (matches the eye)
change-acc ↑	26%	75%	did the *right* cell change
VLM move-acc ↑	17%	42%	InternVL reads the move (ceiling ≈42%)

- Same model, more steps: every metric roughly *doubles*. The early blur was undertraining + information starvation, not a fundamental limit of the approach.
- `change-acc` is our honest metric (SSIM is dominated by the static background). VLM move-acc is capped by InternVL's own reading ability.

How the images come out (hospital, 2000 steps)



SSIM 0.84 / LPIPS 0.13: predicted matches GT: correct player cell, correct HUD counters, crisp background.

`input | PREDICTED | ground truth`.

Latency: real-time-ish on GB10, real-time on A100

Config	ms / frame	fps	notes
GB10, 30 denoise steps	2368	0.42	full quality
GB10, 10 denoise steps	1552	0.64	lighter sampler
A100 (projected)	$\sim$340	$\sim$3	7$\times$ bandwidth (273$\rightarrow$1935 GB/s)
CPU only	$>$115 000	$\ll$0.01	unusable

- Per-frame breakdown (GB10): VAE encode 388 ms + denoise 1068 ms + VAE decode 616 ms. The denoiser dominates: fewer steps = faster.
- Optional Real-ESRGAN upsample: 433 ms on GB10 $\rightarrow$ $\sim$61 ms on A100.
- Bottleneck is memory bandwidth, not FLOPs $\Rightarrow$ A100 gets it to interactive frame rates.

Videos / rollouts (attached files)

- `closed_loop_3way.mp4`: original vs noise-aug vs x-pred+noise-aug, plain closed loop over a full game: the anti-drift progression.
- `neural_engine_video`: teacher-forced vs closed-loop, side by side.
- `play_drift_free`: compositor input | WM render | GT: the drift-free interactive loop (re-anchored each step).

Beamer can't embed video inline: these ship as MP4/GIF alongside the deck in `runs/`. The hospital montage above is the still-frame evidence.

A world model that learned the dynamics of 10 games.

The remaining gap: sharp moving sprites: is a named, fundamental tradeoff with known levers to try next.

References

- Alonso et al. *Diffusion for World Modeling* (DIAMOND), NeurIPS 2024.
- Valevski et al. *Diffusion Models Are Real-Time Game Engines* (GameNGen), 2024.
- Hafner et al. *Mastering Diverse Domains through World Models* (DreamerV3), 2023.
- Blau & Michaeli. *The Perception-Distortion Tradeoff*, CVPR 2018.
- Karras et al. *Elucidating the Design Space of Diffusion Models* (EDM), NeurIPS 2022.
- Diffusion Transformer World-Action Model for AV Scene Prediction, arXiv:2606.12987: reports the same conditional-mean blur.
- Wu et al. Qwen-Image; Real-ESRGAN (Wang et al., ICCVW 2021).